

Health Transformation Review — Operations Guide

For anyone responsible for keeping the platform running: deploying, monitoring, maintaining, and troubleshooting.

Infrastructure Summary

Service	Provider	What It Does
Frontend	Vercel	Next.js app serving all pages and frontend API routes
Backend	Railway	Python FastAPI app powering the AI Analyst
Database	Supabase	PostgreSQL for users, subscriptions, and vector storage
CMS	Sanity	Editorial content management
Payments	Stripe	Subscription billing
Email	Resend	Weekly digest emails
LLM	Groq	AI chat responses
Embeddings	OpenAI	RAG vector embeddings

Deployments

Frontend (Vercel)

Project ID: `prj_LVh0D2YKzL0vQQBGNYpkZSgulcpG`

Auto-deploy: Pushes to `main` branch trigger a production deploy automatically.

Manual deploy:

```
cd frontend
npx vercel --prod
```

Environment variables: Set in the Vercel dashboard under Project → Settings → Environment Variables. All variables from `frontend/.env.local` must be mirrored here for production.

Key production variables to confirm:

- `PYTHON_BACKEND_URL` — must point to the current Railway backend URL
- `NEXT_PUBLIC_APP_URL` — must be the production domain
- `STRIPE_WEBHOOK_SECRET` — must match the webhook endpoint registered in Stripe

Backend (Railway)

Start command: `uvicorn main:app --host 0.0.0.0 --port $PORT`

Health check: `GET /health` every 30 seconds, 10-second timeout.

Restart policy: On failure, up to 3 retries.

Deploy new version:

- Push changes to `backend/main.py` or `requirements.txt`
- Railway auto-deploys from connected Git branch, or trigger manually in Railway dashboard

Environment variables: Set in Railway project → Variables panel. All variables from `backend/.env` must be set here.

Cold start warning: On first deploy or after a restart without an existing pgvector index, the backend builds the RAG index from scratch. This can take 1–3 minutes. During this time, `/api/chat` returns 503. The `/health` endpoint will show `"index_ready": false`.

Monitoring

Is the backend running?

```
curl https://your-backend.railway.app/health
```

Expected response:

```
{
  "status": "ok",
  "index_ready": true,
  "model": "llama-3.3-70b-versatile",
  "embedding_model": "text-embedding-3-small",
  "vector_store": "pgvector",
  "auth_enabled": true
}
```

Problems to look for:

- `index_ready: false` — still building the index. Wait 1–3 minutes. If persists, check Railway logs.
- `vector_store: "local_json"` — `SUPABASE_DB_URL` not set or pgvector connection failing. AI Analyst will work but index won't persist across restarts.
- `auth_enabled: false` — `SUPABASE_JWT_SECRET` not set. Dev mode — all users can access AI regardless of subscription.

Frontend health check route

```
curl https://your-domain.com/api/health
```

This proxies to the backend `/health` endpoint.

Railway logs

Railway dashboard → project → Deployments → click active deployment → Logs tab.

Key log events to monitor:

```
🚀 HTR AI Brain v3 starting...
✅ Loaded existing index from Supabase pgvector    ← fast start
OR
No existing index found – building from scratch... ← slow start
📄 Loading N PDF(s) from data/...
🔗 Fetching Sanity CMS content...
⌚ Embedding N documents...
✅ Index built and stored in Supabase pgvector
```

If you see `Could not init PG vector store`, the backend is falling back to local storage — check `SUPABASE_DB_URL`.

Vercel logs

Vercel dashboard → project → Deployments → click deployment → Functions tab for API route logs.

Stripe events

Stripe Dashboard → Developers → Webhooks → your endpoint → Recent deliveries.

Verify:

- Events are being received (not "Failed to deliver")
- `checkout.session.completed` events show a 200 response

Routine Operations

Publish new content and update the AI knowledge base

1. Publish content in Sanity Studio
2. Content is immediately live on the website (Sanity fetch is not cached)
3. To add it to the AI Analyst's knowledge base:

```
curl -X POST https://your-backend.railway.app/api/ingest \
  -H "Authorization: Bearer $INGEST_SECRET"
```

Or, if you don't have `INGEST_SECRET` set (dev):

```
curl -X POST http://localhost:8000/api/ingest
```

The rebuild runs in the background (1–3 minutes). Monitor Railway logs.

Send the weekly digest email

```
curl -X POST https://your-domain.com/api/digest \
  -H "Authorization: Bearer $DIGEST_SECRET"
```

Response shows how many emails were sent and any errors:

```
{ "message": "Digest sent", "sent": 142, "errors": 0, "total": 142 }
```

What gets sent: The 5 most recently published `policyAnalysis` documents from Sanity.

Recipients: All `subscriber` documents in Sanity with `isActive: true` and `digestEnabled: true`.

Grant a user a subscription manually

Via Supabase SQL editor:

```
-- Grant professional role
INSERT INTO user_roles (user_id, role)
VALUES ('user-uuid-here', 'professional')
ON CONFLICT (user_id, role) DO NOTHING;

-- Remove free role if present
DELETE FROM user_roles
WHERE user_id = 'user-uuid-here' AND role = 'free';
```

Add a new PDF to the AI knowledge base

1. Place the `.pdf` file in `backend/data/`
2. Commit and push (Railway will redeploy)
3. Or if already deployed, trigger re-index manually:

```
curl -X POST https://your-backend.railway.app/api/ingest \
-H "Authorization: Bearer $INGEST_SECRET"
```

No code changes needed — the backend auto-detects all `.pdf` files in the directory.

Rotate API keys

When rotating any API key:

1. Generate the new key in the relevant provider's dashboard
2. Update the environment variable in Railway (backend) or Vercel (frontend)
3. Railway and Vercel apply env var changes on next deploy or via their "redeploy" button
4. Verify with a test request after the new deployment

Keys that may need rotation:

- `OPENAI_API_KEY` — OpenAI dashboard
- `GROQ_API_KEY` — Groq console
- `SANITY_API_TOKEN` — Sanity project settings
- `SUPABASE_SERVICE_ROLE_KEY` — Supabase dashboard → Settings → API
- `STRIPE_SECRET_KEY` — Stripe dashboard → Developers → API keys
- `STRIPE_WEBHOOK_SECRET` — Stripe dashboard → Developers → Webhooks → endpoint → reveal secret
- `RESEND_API_KEY` — Resend dashboard → API keys
- `DIGEST_SECRET` — generate a new random string, update in Vercel env vars

Troubleshooting

AI chat returns "Cannot reach the AI backend"

The Python backend is not running or not reachable.

1. Check `PYTHON_BACKEND_URL` in Vercel env vars — must be the current Railway URL
2. Check Railway dashboard — is the deployment healthy?
3. Check Railway logs for startup errors
4. Test directly: `curl https://your-backend.railway.app/health`

AI chat returns "Access denied"

The user does not have a subscriber+ role, OR `SUPABASE_JWT_SECRET` on the backend doesn't match the Supabase project's JWT secret.

1. Check user's roles in Supabase: `SELECT * FROM user_roles WHERE user_id = 'uuid'`
2. Verify `SUPABASE_JWT_SECRET` on Railway matches the value in Supabase Dashboard → Settings → API → JWT Secret

Stripe checkout doesn't complete / role not granted

1. Check Stripe Dashboard → Developers → Webhooks → recent events
2. Look for `checkout.session.completed` — is it showing 200 or an error?
3. Check Vercel function logs for `/api/stripe/webhook`
4. Verify `STRIPE_WEBHOOK_SECRET` in Vercel matches the Stripe webhook endpoint secret
5. Check `stripe_events` table in Supabase — did the event get logged?
6. Check `user_roles` table — was the role inserted?

Sanity content not appearing on the site

Sanity content is fetched at request time with `useCdn: false`. If content is published but not appearing:

1. Confirm the document is **Published** in Sanity (not just saved as a draft)
2. Confirm the `slug` field is set (most content types require a slug to be queryable)
3. Check for typos in the GROQ query in the relevant page component
4. Check browser devtools Network tab for any failed Sanity API calls

Search returning no results

1. Confirm the document is published in Sanity with a slug
2. Check `/api/search` endpoint directly: `curl "https://your-domain.com/api/search?q=test"`
3. Sanity search uses `match` operator which requires minimum 2 characters

Weekly digest not sending

1. Verify `RESEND_API_KEY` is set in Vercel
2. Verify `DIGEST_SECRET` is set and matches what you're sending
3. Check for active subscribers: `SELECT COUNT(*) FROM subscriber WHERE isActive = true AND digestEnabled = true` (in Sanity dataset)
4. Test with a direct curl to `/api/digest`
5. Check Resend dashboard for send history and errors

pgvector index lost (backend using local_json fallback)

This happens when:

- `SUPABASE_DB_URL` is incorrect or not set
- pgvector extension not enabled in Supabase
- Database password changed

Fix:

1. Enable pgvector: Supabase Dashboard → Database → Extensions → search "vector" → enable
2. Get the correct connection pooler URL: Supabase → Settings → Database → Connection Pooling → Transaction mode (port 6543)
3. Update `SUPABASE_DB_URL` in Railway env vars
4. Redeploy Railway
5. Trigger `POST /api/ingest` to rebuild and store the index in pgvector

Backend running out of memory during index build

LlamaIndex embedding calls are memory-intensive with large document sets.

Options:

- Increase Railway plan memory
- Reduce document count by filtering Sanity queries
- Increase `build_index()` batching (would require code changes)

Scheduled Tasks

Neither Vercel nor Railway provide built-in cron in the current configuration. These operations should be run manually or via an external scheduler:

Task	Frequency	Command
Weekly digest email	Weekly	<code>POST /api/digest</code> with <code>DIGEST_SECRET</code>
RAG re-index (after new content)	After publishing	<code>POST /api/ingest</code> with <code>INGEST_SECRET</code>

To automate with a cron service (e.g., cron-job.org, GitHub Actions, Vercel Cron):

Digest example (GitHub Actions):

```
on:
  schedule:
    - cron: '0 9 * * 1' # Every Monday at 9am UTC
jobs:
  digest:
    runs-on: ubuntu-latest
    steps:
      - run: |
          curl -X POST ${{ secrets.SITE_URL }}/api/digest \
            -H "Authorization: Bearer ${{ secrets.DIGEST_SECRET }}"
```

Environment Variables Checklist

Vercel (Frontend)

Variable	Required	Notes
NEXT_PUBLIC_SANITY_PROJECT_ID	Yes	fxz10xl7
NEXT_PUBLIC_SANITY_DATASET	Yes	production
NEXT_PUBLIC_SANITY_API_VERSION	Yes	2023-10-01
SANITY_API_TOKEN	Yes	Read/write token
NEXT_PUBLIC_SUPABASE_URL	Yes	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Yes	Public anon key
SUPABASE_SERVICE_ROLE_KEY	Yes	Secret — server only
STRIPE_SECRET_KEY	Yes	sk_live_...
STRIPE_WEBHOOK_SECRET	Yes	whsec_...
STRIPE_PRICE_SUBSCRIBER_MONTHLY	Yes	Stripe price ID
STRIPE_PRICE_SUBSCRIBER_YEARLY	Yes	Stripe price ID
STRIPE_PRICE_STUDENT_MONTHLY	Yes	Stripe price ID
STRIPE_PRICE_STUDENT_YEARLY	Yes	Stripe price ID
STRIPE_PRICE_PROFESSIONAL_MONTHLY	Yes	Stripe price ID
STRIPE_PRICE_PROFESSIONAL_YEARLY	Yes	Stripe price ID
RESEND_API_KEY	Yes	re_...
DIGEST_SECRET	Yes	Random secret string
PYTHON_BACKEND_URL	Yes	Railway backend URL
NEXT_PUBLIC_APP_URL	Yes	Production frontend URL

Railway (Backend)

Variable	Required	Notes
OPENAI_API_KEY	Yes	For embeddings
GROQ_API_KEY	Yes	For LLM
SANITY_PROJECT_ID	Yes	fxz10xl7
SANITY_DATASET	No	Default: production

Variable	Required	Notes
SANITY_API_TOKEN	Yes	Read token
SUPABASE_URL	Yes	Supabase project URL
SUPABASE_SERVICE_ROLE_KEY	Yes	Service role key
SUPABASE_JWT_SECRET	Yes	From Supabase Settings → API
SUPABASE_DB_URL	Yes	PostgreSQL pooler URL (port 6543)
FRONTEND_URL	Yes	Production frontend URL (for CORS)
INGEST_SECRET	Recommended	Protects <code>/api/ingest</code>
GROQ_MODEL	No	Default: <code>llama-3.3-70b-versatile</code>

Vercel Webhook Registration (Stripe)

In the Stripe dashboard:

1. Developers → Webhooks → Add endpoint
2. URL: `https://your-domain.com/api/stripe/webhook`
3. Events to listen for:
 - `checkout.session.completed`
 - `customer.subscription.updated`
 - `customer.subscription.deleted`
 - `invoice.payment_failed`
4. Copy the signing secret → set as `STRIPE_WEBHOOK_SECRET` in Vercel

Railway CORS Configuration

The backend only allows requests from:

- `FRONTEND_URL` env var value
- `http://localhost:3000` (always allowed)

If the frontend URL changes (e.g., new Vercel domain), update `FRONTEND_URL` in Railway and redeploy.

First-Time Production Setup

If setting up from scratch:

1. **Supabase:** Create project → enable pgvector extension → run database migrations to create tables (`profiles`, `user_roles`, `subscriptions`, `stripe_customers`, `stripe_events`) → set up RLS policies
2. **Sanity:** Create project `fxz10xl7` (or existing) → deploy studio → create API token with editor access
3. **Stripe:** Create products and prices matching the four plan tiers → register webhook endpoint → copy price IDs
4. **Resend:** Create API key → verify sending domain (`healthtransformationreport.com`)
5. **Railway:** Create project → connect Git repo → set all backend env vars → deploy
6. **Vercel:** Connect frontend directory → set all frontend env vars → deploy
7. **AI index:** After backend deploys and Sanity has content, trigger `POST /api/ingest`
8. **Test:** Create a test user → subscribe via Stripe test mode → verify role granted → test AI chat